Module 6 Notes: Fun4All, Software Stack and Simulation

Reading notes for Module06_Fun4All_and_Simulation.pdf. Questions I asked while reading, with answers. Grouped by module section; sections with no questions are omitted. Also available as a PDF (Module06_Notes.pdf).

---

4. Your analysis module: the SubsysReco lifecycle

Q: What is a SubsysReco?

A: It's the base class that every Fun4All module inherits from: reconstruction, calibration, Geant4 (`PHG4Reco`), and my own analysis module (`pi0EtaAnalysisPP24`). It defines a fixed set of virtual methods with do-nothing defaults: `Init`, `InitRun`, `process_event`, `ResetEvent`, `EndRun`, `End`. You inherit from it and override only what you need:

```
class ClusterCounter : public SubsysReco {
 public:
  ClusterCounter(const std::string &name = "ClusterCounter")
    : SubsysReco(name) {}
  int Init(PHCompositeNode *topNode) override;
  int process_event(PHCompositeNode *topNode) override;
  int End(PHCompositeNode *topNode) override;
};
```

Why it matters: every module has the same interface, so the `Fun4AllServer` only needs a list of SubsysReco* in registration order. For each event it calls `process_event` on each one (polymorphism). Each method returns a code (`EVENT_OK`, `ABORTEVENT`, …) that tells the server whether to continue. Modules never call each other; they only communicate through the node tree.

Q: What is `Init`?

A: It's the first SubsysReco method, called once before any events are read. It's for one-time setup: book histograms, open output files/TTrees, create your own output nodes (nodes can't be added to DST mid-run).

- **Not** for run-dependent things. No event has been read yet, so the run number is unknown. Calibrations, hot-tower maps and trigger config go in `InitRun`.
- Call order: constructor → setters (in the macro) → `Init` → `InitRun` → `process_event` … → End.
- The constructor should only store defaults. It runs before your setters, so setup belongs in `Init`, where the macro's options have been applied.
- Return `EVENT_OK`, or an abort code if setup fails so the job stops early.

Q: What do "setter" and "getter" mean here (`m_nodeName` "with a setter, so the same module can count clusters from different builders")?

A: These are plain C++ terms. A setter is a public method that sets a private member from outside the class, and a getter returns it. `m_nodeName` holds the name of the node that `process_event` reads. The setter means the name is chosen in the macro, not hard-coded:

```
// in ClusterCounter.h
void set_nodeName(const std::string &n) { m_nodeName = n; }     // setter
const std::string &get_nodeName() const { return m_nodeName; }  // getter
std::string m_nodeName = "CLUSTERINFO_CEMC";  // private, with default

// in the macro
auto *cc = new ClusterCounter("CountTopo");
// example name; check NODETREE for the real names
cc->set_nodeName("TOPOCLUSTER_ALLCALO");
se->registerSubsystem(cc);
```

Each cluster builder writes to a differently named node, so one compiled module can count any of them, and you can even register it twice with different names, without recompiling. (The `m_` prefix is the sPHENIX convention for member variables.)

5. Schema evolution

Q: What is a "sentinel" (base-class getters "return a sentinel")?

A: A sentinel is a special return value that means "no real value here". If a class version doesn't store a field (e.g. `TowerInfov1` has no chi2), the base-class default getter returns a placeholder:

```
virtual float get_chi2() { return NAN; }  // in TowerInfo (schematic)
```

- A good sentinel can't be mistaken for real data. `NaN` is ideal for floats (test with `std::isnan`). 0 is weak because it can look like a real value.
- Other common ones: `-1` or `-999`, `nullptr` (e.g. `getClass` returns `nullptr` for a missing node), `std::string::npos`.
- Practical point: a spike at 0 or NaNs in a `get_time()`/`get_chi2()` histogram is not bad data. It's a version that doesn't store the field, or a zero-suppressed tower with no fit (Module 7). Check before using the value.

6. Input and output managers

Q: What is an input manager?

A: It reads files and puts their contents onto the node tree, one event at a time. Modules never open files; they `getClass` objects that the input manager has already placed on the tree. It's step 1 of the server's per-event loop, before any `process_event` is called.

```
// the name "DSTCALO" is just a label
Fun4AllInputManager *in = new Fun4AllDstInputManager("DSTCALO");
// list file: one path per line (or AddFile for a single file)
in->AddListFile("dst_calo.list");
se->registerInputManager(in);
```

- The type matches the file: `Fun4AllDstInputManager` (DSTs, the analysis default), `Fun4AllPrdfInputManager` (raw), `Fun4AllHepMCInputManager` (generator events), `Fun4AllRunNodeInputManager` (RUN node only), `Fun4AllNoSyncDstInputManager` (no event matching), `Fun4AllHepMCPileupInputManager` (pile-up).

- With several managers, the server synchronises them: an event reaches the modules only if all of them have the same run and event number. A segmentation mismatch means zero events processed.
- Output managers do the reverse: they take nodes from the tree and write them to a file.

General / cross-cutting notes

Q: What does a pointer do (C++ refresher)?

A: A pointer is a variable that holds a memory address: where an object lives, not the object itself.

```
int x = 5;
int *p = &x;   // & = "address of"; p holds x's address
*p = 7;        // * = dereference: change what's there (x is now 7)
towers->size();  // -> = follow pointer, use member; = (*towers).size()
```

Why Fun4All uses pointers everywhere:

1. **No copying:** `getClass` returns the address of the container on the node tree, so you work on the real object in place. (`auto *` = "a pointer to whatever type this returns".)
2. **Polymorphism:** a `TowerInfo*` can point at a `TowerInfov4`, and `t->get_energy()` runs the v4 version at run time. The same mechanism lets the server hold `SubsysReco*`s and call `process_event` on each. This is what makes schema evolution work.
3. **`nullptr` = "nothing found":** this is a sentinel. Dereferencing `nullptr` crashes the job (segfault), so always check `if (!towers) ...`.

Ownership: `new` creates an object and returns a pointer to it, and the owner must eventually `delete` it. Objects on the node tree, and modules handed to `registerSubsystem`, are owned by Fun4All, so never `delete` them yourself.